Question 6 — Basic Jenkins CI/CD Pipeline

Student: Kartik Roll No.: 2301730205

1. Aim

Create a Jenkins pipeline job that uses the Jenkinsfile in the repository to automatically checkout, build, test, package and deploy the Flask application.

2. Job Configuration

Setting	Value
Job type	Multibranch / Pipeline
Definition	Pipeline script from SCM
SCM	Git
Repository URL	https://github.com/Kartik/devops-lab-1
Branch	*/main
Script path	Jenkinsfile
Build trigger	Poll SCM — * * * * * (every minute)

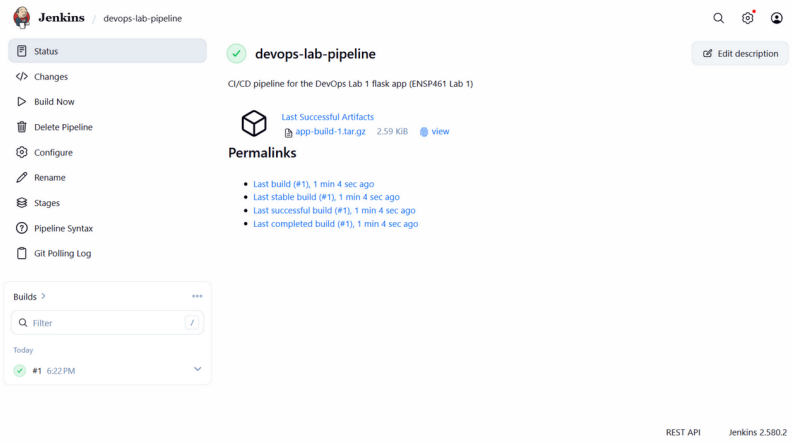


Fig: Jenkins pipeline job created and configured

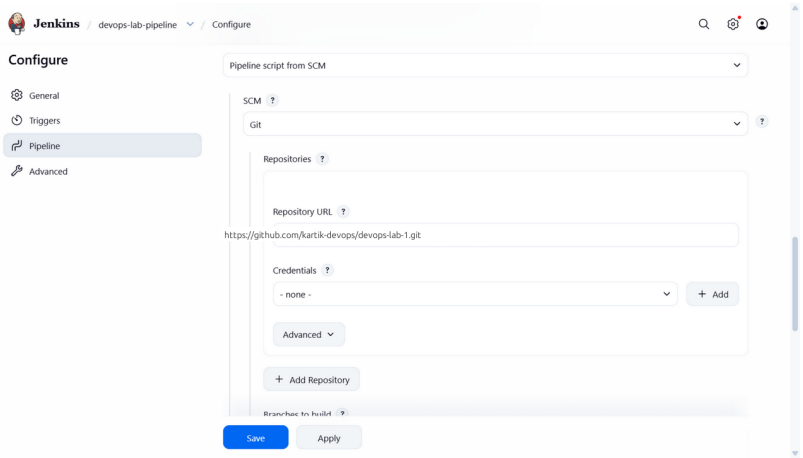


Fig: SCM configuration — Git repository and branch

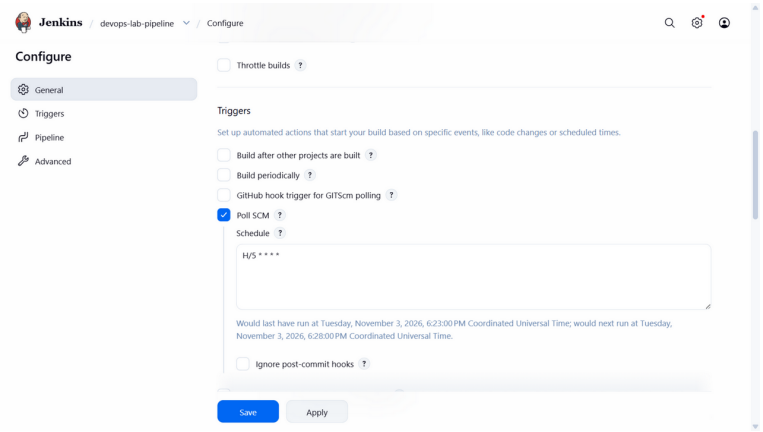


Fig: Build trigger — Poll SCM every minute

3. Build #1 — First Manual Execution

Build #1 was triggered manually using Build Now to verify the full pipeline. All five stages completed successfully.

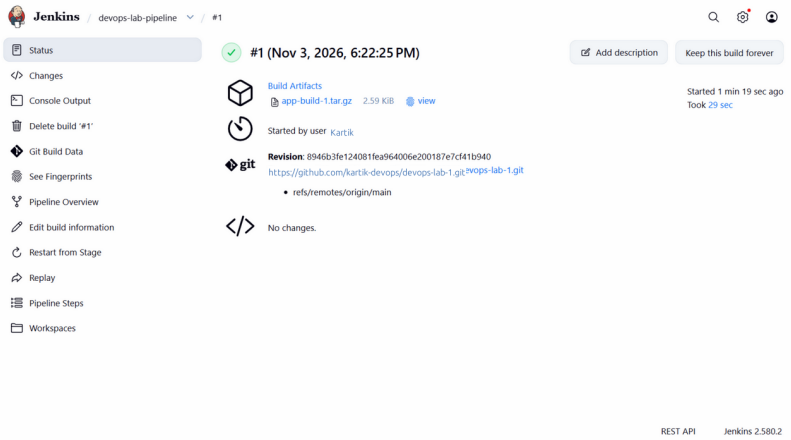


Fig: Build #1 stage view — all stages passed

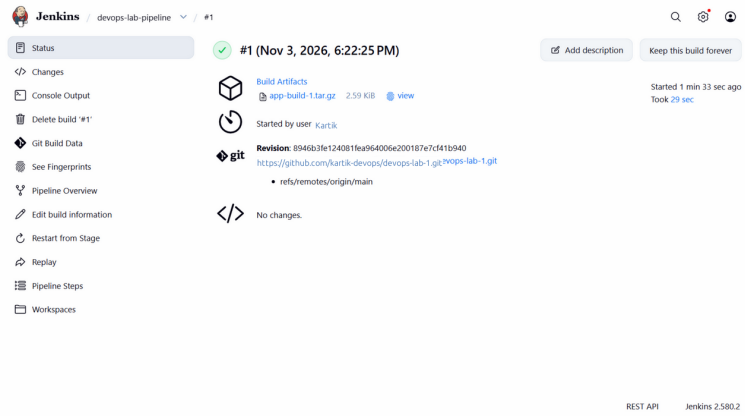


Fig: Build #1 pipeline overview — SUCCESS

4. Console Output — Build #1

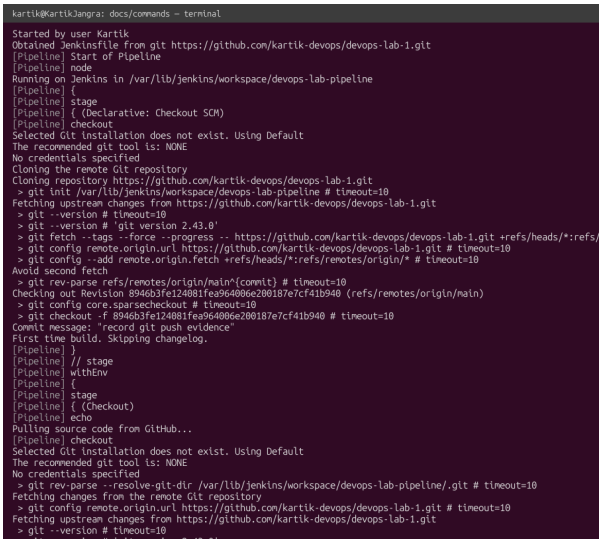


Fig: Console — Checkout stage

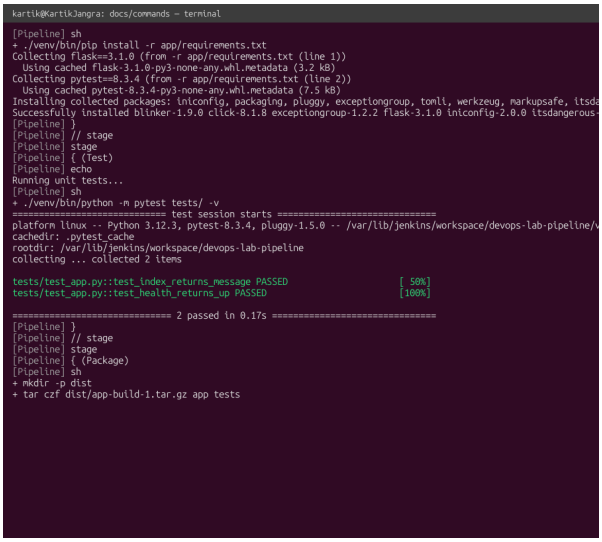


Fig: Console — Build stage (virtualenv + pip install)

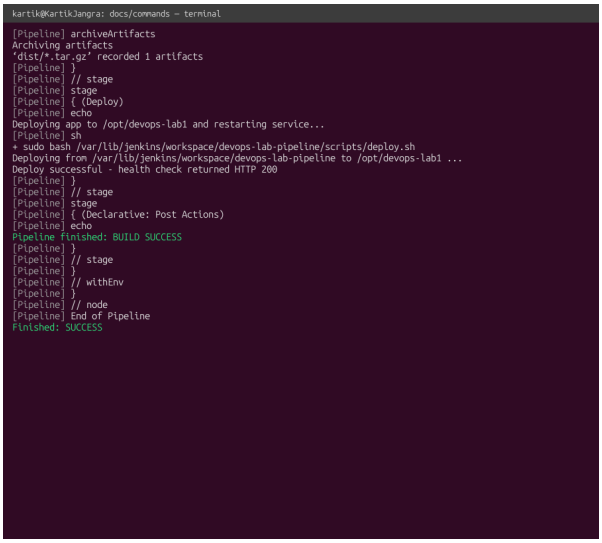


Fig: Console — Test stage (pytest)

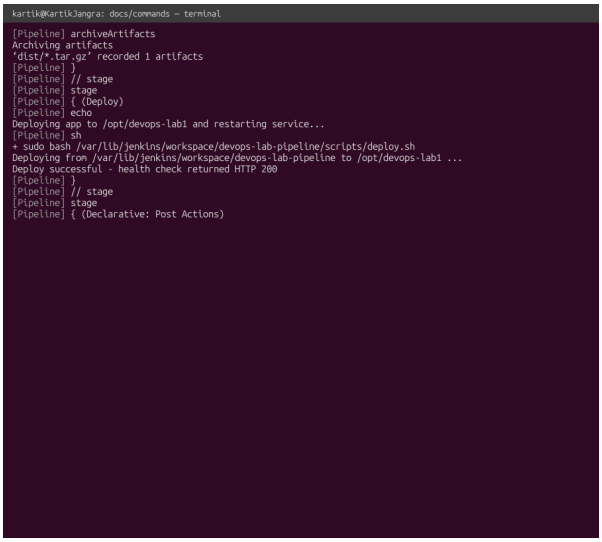


Fig: Console — Package stage (tar artefact)



Fig: Console — Deploy stage & Finished: SUCCESS

5. Observations

- All five pipeline stages — Checkout, Build, Test, Package, Deploy — passed on Build #1.
- The pip install step installed Flask 3.1.0 into an isolated virtualenv.
- pytest reported 2 tests passed with no warnings.
- The deploy script confirmed HTTP 200 from the /health endpoint after deployment.
- Total build duration was approximately 29 seconds.

6. Conclusion

A fully functional Jenkins CI/CD pipeline was created using a declarative Jenkinsfile. Build #1 validated the entire pipeline end-to-end, confirming that every stage executes correctly and the application is deployed and

healthy after each run.